
CHAPTER 5

LECTURE 6: ADVERSARIAL SEARCH/GAMES

Note: This lecture closely follows (sometimes, directly borrowed from) Chapter 5 to 5.2.4 of Russel and Norvig, *Artificial Intelligence: A Modern Approach*.

5.1 Introduction

Now we would like to focus on competitive environments, in which two or more agents have conflicting goals. The agents in these environments work against each other, minimizing the evaluation value of each other. To tackle such environments, we use Adversarial search. We will study Minimax search, using which we can find the optimal move for an agent in the restricted game environment. We will also study alpha-beta pruning where including pruning we can make the search more efficient by ignoring portions that do not help us find the optimal solution.

Instead of handling real life problems which have a lot of uncertainty and are difficult to handle we focus on games. Examples of such games are chess, Go, poker etc. We will use much easier games than these to demonstrate the adversarial nature.

5.1.1 Two player zero-sum games

- **Deterministic:** We can determine the outcome of an action.
- **Two players:** Only two agents working against each other. Example: Player A and Player B.
- **Turn taking:** Agents take turns alternatively. Example: After Agent (Player) A plays a move (chooses their action), Agent (Player) B will get to play their move (choose their action) and so on.
- **Perfect Information:** Fully observable environment, as in there are no parts of the game unknown to the agents.

- **Zero-Sum Games:** The total sum of points of the players in the game is zero. If one player wins (+1 points) that would mean the opponent player loses (-1 points). The total sum of the points in the game will be zero. When there is a draw, the point for both players is zero, meaning the sum will be zero.

We will use the term **move** as a synonym for **action**, and **position** as a synonym for **state**.

5.1.2 Maximizer and Minimizer

Continuing from the concept of a zero-sum game, let us refer to the two players (agents) of the game as MAX and MIN.

MAX aims to maximize its own game points. In a zero-sum game, the total payoff for all players is constant, meaning one player's gain is exactly equal to another's loss. Therefore, MAX seeks to achieve the highest possible outcome for itself, knowing that this will occur at the expense of the other player.

MAX evaluates all possible moves by considering what its opponent might choose, countering their moves. It chooses the option that leads to the highest evaluated payoff under optimal play.

On the other hand, MIN aims to win by minimizing MAX's points. While choosing its move, MIN selects the move that will reduce the good options for MAX, limiting MAX's game points and forcing it towards an outcome where MAX's game points are minimal. This strategy minimizes MIN's loss as well.

For example, in a simple game like tic-tac-toe, MAX aims to align three of its marks in a row, thereby maximizing its chance of winning and ensuring the other player's loss. Conversely, MIN attempts to block MAX's efforts to align three marks, while also seeking opportunities to create a threat that MAX must respond to, thereby steering the game towards a draw or a win for MIN.

MAX and MIN alternatively choose moves, each striving to reach their respective goals. The minimax algorithm provides a way to determine the best possible move for a player (assuming both players play optimally) by minimizing the possible loss for a worst-case (maximum loss) scenario.

These concepts are crucial in designing agents capable of thinking several steps ahead, anticipating and countering the opponent's strategies effectively.

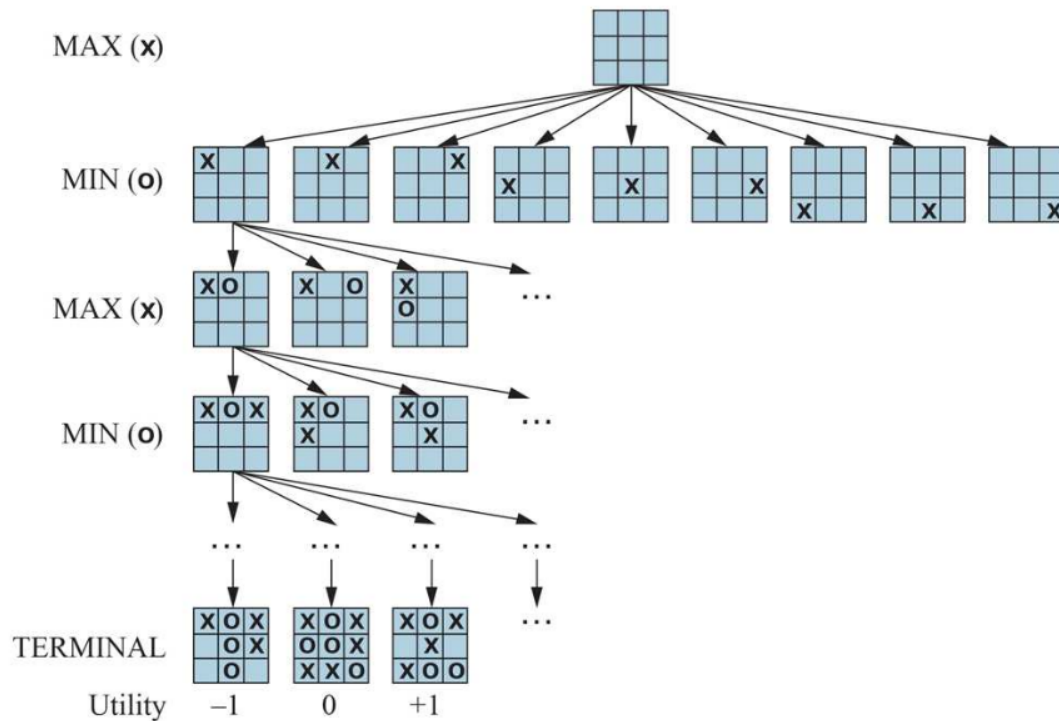
The following elements formally define the game:

- S_0 : The initial state, which specifies how the game is set up at the start.
- $TO-MOVE(s)$: The player whose turn it is to move in state s .
- $ACTIONS(s)$: The set of legal moves in state s .

- **RESULT(s , a)**: The transition model, specifying the state that results from executing action a in state s .
- **IS-TERMINAL**: A test to determine if state s is terminal, returning true if the game has concluded and false otherwise. States where the game has ended are called terminal states. Example, Win, Lose, Draw.
- **UTILITY**: A utility function (also known as an objective function or payoff function), which provides the final numerical value to player p when the game concludes in a terminal state s . For instance, in chess, the result can be a win, loss, or draw, with values 1, 0, or $\frac{1}{2}$ respectively. In some games, the payoff range can be broader, such as in backgammon, where it ranges from 0 to 192.

The initial state, the action function, and the result function together define the state space graph. In this graph, the vertices represent states and the edges represent moves. A state can be reached by multiple paths. From the search tree, we can construct the game tree that traces every sequence of moves all the way to the terminal state.

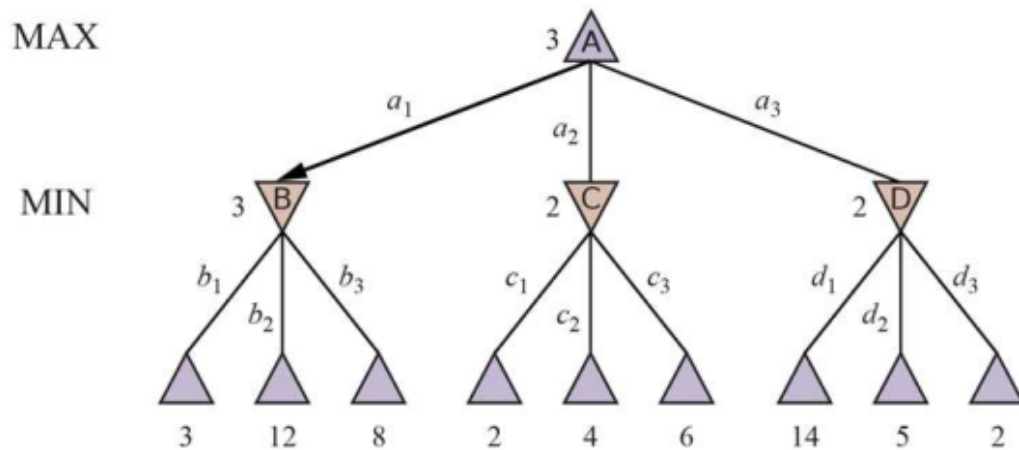
The figure below illustrates part of the game tree for tic-tac-toe (noughts and crosses). Starting from the initial state, MAX has nine possible moves. Players alternate turns, with MAX placing an X and MIN placing an O, until reaching terminal states where either one player has three in a row or the board is completely filled. The number on each leaf node represents the utility value of the terminal state from MAX's perspective; higher values benefit MAX and disadvantage MIN (hence the players' names).



A (partial) game tree for the game of tic-tac-toe. The top node is the initial state, and MAX moves first, placing an x in an empty square. We show part of the tree, giving alternating moves by MIN (o) and MAX (x), until we eventually reach terminal states, which can be assigned utilities according to the rules of the game.

For tic-tac-toe, the game tree is relatively small—fewer than $9! = 362,880$ terminal nodes (with only 5,478 distinct states). But for chess, there are over 10^{40} nodes, so the game tree is best thought of as a theoretical construct that we cannot realize in the physical world.

5.2 Making optimal decisions using Minimax Search



A two-ply game tree. The $\triangle$ nodes are "MAX nodes," in which it is MAX's turn to move, and the ∇ nodes are "MIN nodes." The terminal nodes show the utility values for MAX; the other nodes are labeled with their minimax values. MAX's best move at the root is a_1 , because it leads to the state with the highest minimax value, and MIN's best reply is b_1 , because it leads to the state with the lowest minimax value.

Consider the trivial game in Figure. The possible moves for MAX at the root node are labeled a_1 , a_2 , a_3 , and so on. The possible replies to for MIN are b_1 , b_2 , b_3 , and so on. This particular game ends after one move each by MAX and MIN.

NOTE: In some games, the word "move" means that both players have taken an action; therefore the word *ply* is used to unambiguously mean one move by one player, bringing us one level deeper in the game tree. The utilities of the terminal states in this game range from 2 to 14.

Given a game tree, the optimal strategy can be determined by working out the **minimax value** of each state in the tree, which we write as $\text{MINIMAX}(s)$. The minimax value is the utility (for MAX) of being in that state, *assuming that both players play optimally* from there to the end of the game. The minimax value of a terminal state is just its utility. In a non-terminal state, MAX prefers to move to a state of maximum value when it is MAX'S turn to move, and MIN prefers a state of minimum value (that is, minimum value for MAX and thus maximum value for MIN). So we have:

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{if IS-TERMINAL}(s) \\ \max_{a \in \text{Actions}} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MIN} \end{cases}$$

5.3 Minimax algorithm

The minimax algorithm explores the game tree using a depth-first search. If the tree's maximum depth is m and there are b legal moves at each point, the time complexity is $\mathcal{O}(b^m)$

and the space complexity is $\mathcal{O}(bm)$.

The exponential complexity makes minimax impractical for complex games. For instance, chess has a branching factor of about 35, and the average game has a depth of about 80 plies. This means we would need to search almost 10^{123} states, which is not feasible.

Algorithm 6 MINIMAX-SEARCH

```
function MINIMAX-SEARCH(game, state) returns an action
  player  $\leftarrow$  game.TO-MOVE(state)
  value, move  $\leftarrow$  MAX-VALUE(game, state)
  return move
```

```
function MAX-VALUE(game, state) returns a (utility,move) pair
  if game.IS-TERMINAL(state) then return game. UTILITY(state,player),null
   $v \leftarrow -\infty$ 
  for each a in game. ACTIONS(state) do
     $v2, a2 \leftarrow$  MIN-VALUE(game, game.RESULT(state,a))
    if  $v2 > v$  then
       $v, \text{move} \leftarrow v2, a$ 
   $v, \text{move}$ 
```

```
function MIN-VALUE(game, state) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state,player),null
   $v \leftarrow +\infty$ 
  for each a in game.ACTIONS(state) do
     $v2, a2 \leftarrow$  MAX-VALUE(game, gmae.RESULT(state,a))
    if  $v2 < v$  then
       $v, \text{move} \leftarrow v2, a$ 
  return  $v, \text{move}$ 
```

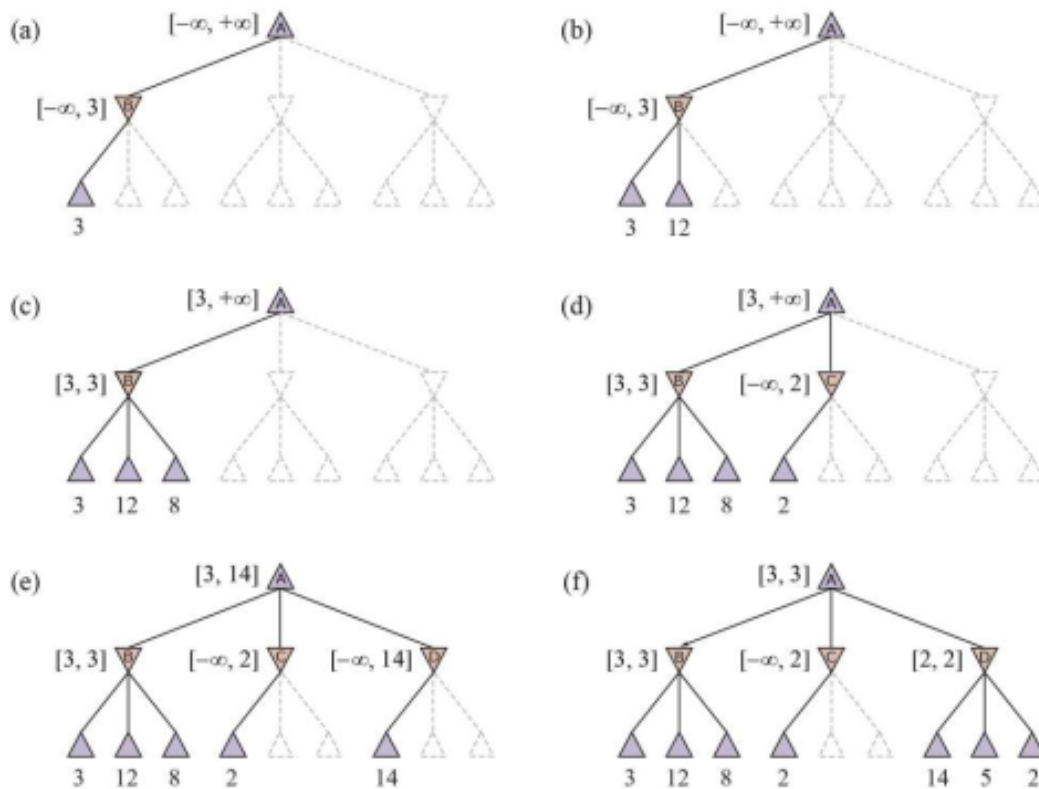
5.4 Alpha—Beta Pruning

Alpha-beta pruning is an optimization technique for the minimax algorithm, which is used in decision-making processes for two-player games like chess or tic-tac-toe. The primary goal of alpha-beta pruning is to reduce the number of nodes evaluated in the game tree, thereby improving the efficiency of the minimax algorithm.

In the minimax algorithm, every possible move and counter-move is explored to determine the optimal strategy. However, this exhaustive search can become computationally expensive, especially for complex games with large search spaces. Alpha-beta pruning addresses this issue by eliminating branches in the game tree that do not influence the final decision.

Let's revisit the two-ply game tree from the previous figure. This time, we will carefully

track what we know at each step. The steps are detailed in the figure below. The result is that we can determine the minimax decision without needing to evaluate two of the leaf nodes.



Stages in calculating the optimal decision for the game tree in the previous figure are shown below. At each point, we display the range of possible values for each node.

- The first leaf under B has a value of 3. Thus, B , a MIN node, has a value of at most 3.
- The second leaf under B has a value of 12. Since MIN would avoid this move, B 's value remains at most 3.
- The third leaf under B has a value of 8. After evaluating all of B 's successor states, B 's value is exactly 3. This means the root value is at least 3, because MAX can choose a move worth 3 at the root.
- The first leaf under C has a value of 2. Therefore, C , a MIN node, has a value of at most 2. Since B is worth 3, MAX would never choose C . Hence, there is no need to examine the other successors of C . This demonstrates alpha-beta pruning.
- The first leaf under D has a value of 14, making D worth at most 14. This is still higher than MAX's best alternative (i.e., 3), so we need to continue exploring D 's successors. Now, we also know the root's value is at most 14.

- (f) The second successor of D has a value of 5, so we continue exploring. The third successor is worth 2, so D 's exact value is 2. MAX's decision at the root is to move to B , yielding a value of 3.

Another way is to see this as a simplification of the MINIMAX formula. Consider the two unevaluated successors of node C in the figure above. Let these successors have values x and y . Then the value of the root node is given by

$$\begin{aligned}
 \text{MINIMAX}(\text{root}) &= \max(\min(3, 12, 8), \min(2, x, y), \min(14, 5, 2)) \\
 &= \max(3, \min(2, x, y), 2) \\
 &= \max(3, z, 2) \quad (\text{where } z = \min(2, x, y) \leq 2) \\
 &= 3
 \end{aligned}$$

In other words, the value of the root and hence the minimax decision are *independent* of the values of the leaves x and y and therefore they can be pruned.

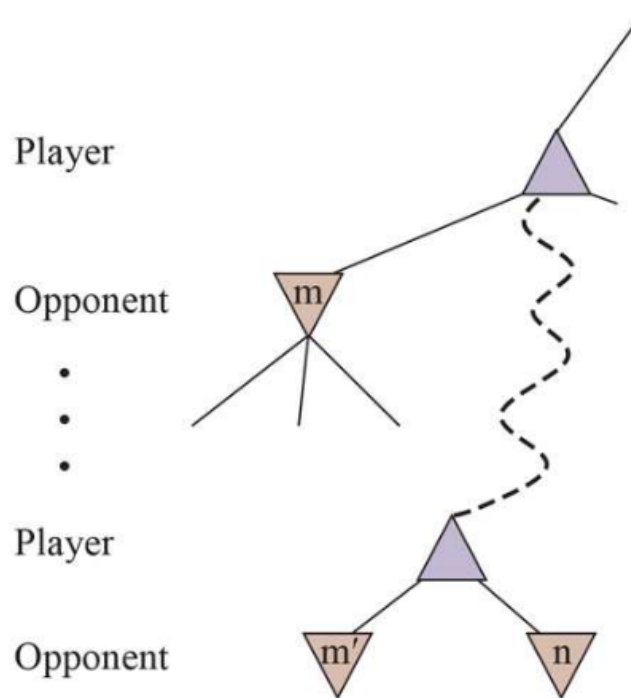
Algorithm 7 Alpha-beta Search Algorithm

function ALPHA-BETA-SEARCH(game, state) **returns** an action
 player $\leftarrow$ TO-MOVE(state)
 value, move $\leftarrow$ MAX-VALUE(game, state, $-\infty$, $+\infty$)
return move

function MAX-VALUE(game, state, α , β) **returns** a (utility, move) pair
if game.IS-TERMINAL(state) **then return** game.UTILITY(state, player), null
 $v \leftarrow -\infty$
for each a **in** game.ACTIONS(state) **do**
 $v2, a2 \leftarrow$ MIN-VALUE(game, game.RESULT(state, a), α , β)
 if $v2 > v$ **then**
 $v, move \leftarrow v2, a$
 $\alpha \leftarrow \text{MAX}(\alpha, v)$
 if $v \geq \beta$ **then return** $v, move$
return $v, move$

function MIN-VALUE(game, state, α , β) **returns** a (utility, move) pair
if game.IS-TERMINAL(state) **then return** game.UTILITY(state, player), null
 $v \leftarrow +\infty$
for each a **in** game.ACTIONS(state) **do**
 $v2, a2 \leftarrow$ MAX-VALUE(game, game.RESULT(state, a), α , β)
 if $v2 < v$ **then**
 $v, move \leftarrow v2, a$
 $\beta \leftarrow \text{MIN}(\beta, v)$
 if $v \leq \alpha$ **then return** $v, move$
return $v, move$

Alpha-beta pruning can be used on trees of any depth, and it often allows for pruning entire subtrees, not just leaves. The basic principle is this: consider a node n in the tree (see Figure below) where Player can choose to move to n . If Player has a better option either at the same level (e.g., m' in the figure below) or higher up in the tree (e.g., m in the figure below), Player will not move to n . Once we learn enough about n by examining some of its successor state to make this decision, we can prune n .



The general case for alpha-beta pruning. If m or m' is better than n for Player, we will never get to n in play.

Minimax search uses a depth-first approach, so we only consider the nodes along a single path in the tree at any given time. Alpha-beta pruning uses two additional parameters in $\text{MAX-VALUE}(\text{state}, \alpha, \beta)$, which set bounds on the backed-up values along the path.

α = the value of the best (i.e., highest-value) choice we have found so far at any choice point along the path for MAX. Think: α = “at least.”

β = the value of the best (i.e., lowest-value) choice we have found so far at any choice point along the path for MIN. Think: β = “at most”.

As the search progresses, these values are updated to reflect the highest and lowest scores that MAX and MIN can achieve, respectively. When a move is found that makes a branch less favorable than previously examined branches, that branch is pruned, meaning it is not explored further.

This pruning does not affect the final result of the minimax algorithm but significantly

reduces the number of nodes evaluated, making it more efficient. In the best case, alpha-beta pruning can reduce the time complexity from $\mathcal{O}(b^m)$ to $\mathcal{O}(b^{\frac{m}{2}})$ where b is the branching factor and m is the maximum depth of the tree.

Alpha-beta pruning enables more effective decision-making in complex games, allowing for deeper searches and better strategic planning within practical time limits.

5.4.1 Worst Case Scenario for Alpha-Beta Pruning

In the worst-case scenario of alpha-beta pruning, the algorithm does not effectively prune any branches, and it ends up exploring the entire game tree, similar to a standard minimax search. This situation can arise when the nodes are ordered in the least optimal way for pruning.

Key Points

- **Branch Ordering:** The worst case occurs when the best moves are always considered last. Alpha-beta pruning relies on evaluating the most promising moves first to maximize pruning. If the least promising moves are evaluated first, fewer branches are pruned.
- **No Pruning:** When the algorithm does not prune any branches, it evaluates every possible move at each depth level, leading to the maximum number of nodes being explored.
- **Time Complexity:** In the worst case, the time complexity of alpha-beta pruning is the same as that of the minimax algorithm without pruning, which is $\mathcal{O}(b^m)$. Here, b is the branching factor (the number of legal moves at each point), and m is the maximum depth of the tree.

Improving alpha-beta pruning

In complex games with large search spaces, to improve the efficiency and effectiveness of the alpha-beta pruning these following techniques can be used.

- **Move-Ordering:** Move-ordering prioritizes evaluating the most promising moves first, increasing the chances of effective pruning in alpha-beta search. This is usually achieved by sorting moves based on heuristic evaluations or previous search results, aiming to maximize the number of branches pruned.
- **Killer Moves:** Killer moves are specific moves that have previously caused significant pruning in similar positions and are tried early in the search to maximize pruning opportunities. These moves are stored and reused, assuming that moves which were effective in the past are likely to be effective again.
- **Iterative Deepening:** Iterative deepening involves repeatedly running depth-limited searches with increasing depth limits, combining the benefits of depth-first search (using

less memory) and breadth-first search (finding the optimal solution). It provides a way to use the best move from shallow searches to improve move-ordering in deeper searches.

- **Heuristic Alpha-Beta:** Heuristic alpha-beta uses evaluation functions to estimate the value of non-terminal nodes, guiding the search and pruning process more effectively based on heuristics. These evaluation functions consider factors like material balance, positional advantages, and other domain-specific knowledge to approximate the true minimax value of a position.